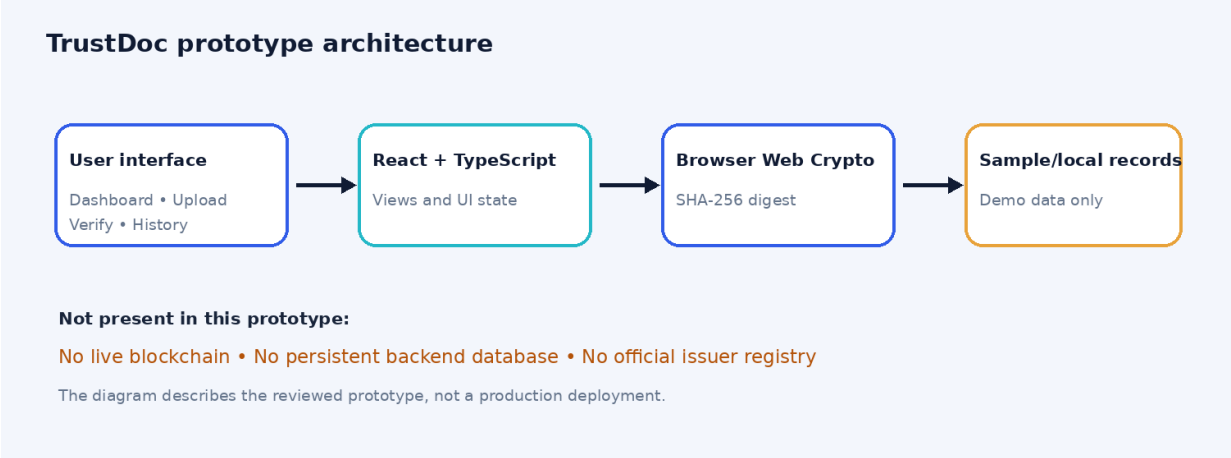
TRUSTDOC

Blockchain-Inspired Government Document Verification System

PROJECT REPORT

Emerging Technologies Coursework
Emerging Technology for a Smarter Future



A student prototype exploring browser-based SHA-256 hashing and reference-based document checking

Bhanuka Hewawasam • KUHNDSE261F-007
Semitha Binuk • KUHNDSE261F-009
Dulaj Adhikari - KUHNDSE261F-011

Repository: github.com/Bhanuka-Hewawasam/ET_BlockChain

Abstract

TrustDoc is a front-end student project that shows how documents could be checked against reference records through a simple digital process. The prototype includes a dashboard, a certificate-upload page, a reference-based verification page and a sample verification-history page. It also uses browser-side SHA-256 hashing to demonstrate how a file can be represented by a unique digital fingerprint.

The idea behind the project comes from two related concepts: using cryptographic hashes to check file integrity and using linked hashes in blockchain systems. However, the current application is not connected to a live blockchain or an official government registry, and it does not compare every uploaded file with a complete trusted hash. At this stage, verification is based on sample or locally stored reference records. This report explains what the prototype currently does and outlines what would be required to develop it into a dependable real-world system.

Report roadmap

Section	What it explains
1–3. Context and objectives	The problem, aim, objectives and scope of TrustDoc.
4–5. Technology	React, TypeScript, Vite, SHA-256 and the blockchain-inspired concept.
6–7. Design and implementation	The screens, intended workflow and current behaviour.
8–9. Prototype review	Screenshots, observations, limitations and security considerations.
10–12. Future work and conclusion	Recommended improvements, final summary and technical references.

Key terms

Term	Simple explanation
Document integrity	Confidence that a file's contents have not changed from a trusted version.
Cryptographic hash	A fixed-length fingerprint calculated from data.
SHA-256	A hash algorithm that produces a 256-bit digest.
Reference record	A stored record found using a document reference number.
Blockchain-inspired	Using concepts associated with linked, hash-based records without claiming a live blockchain.

1. Introduction

Educational certificates, training records and other official documents are now commonly shared in digital form. When someone receives one of these files, they may need to confirm that it matches a record issued by a trusted organisation. In many cases, this still involves contacting the issuer directly and waiting for a response.

A digital verification service could make this process quicker and more convenient. For example, a user could enter a document reference number and view the status linked to that record. Cryptographic hashing could provide an additional check by comparing the uploaded file with the securely stored digest of the original file.

We developed TrustDoc as a student prototype to present this idea through a simple web interface. The application has four main user areas: the dashboard, certificate upload, document verification and verification history. Throughout this report, we separate the features that are already visible in the prototype from the functions that would still need to be developed for a production system.

2. Problem statement

A digital document on its own may not give the receiver enough evidence to confirm that it matches a trusted original or was issued by the organisation shown on it. A useful verification process therefore needs a trusted source of reference information, a clear checking method and careful wording when results are displayed.

TrustDoc explores a simple user journey for addressing this issue. The present version should be viewed as a concept demonstration rather than a complete authenticity service. It searches for a reference in sample or local records, but it does not yet compare a newly uploaded file with a complete trusted SHA-256 digest.

3. Aim and objectives

Aim: To build and demonstrate a simple digital approach for checking a document through its file hash and a stored reference record.

- Allow the user to choose a supported document or image file through a simple interface.
- Generate the SHA-256 digest of the selected file directly in the browser.
- Display document references and example verification details in a way that is easy to understand.
- Include a dashboard summary and an example page showing previous verification attempts.
- Demonstrate the idea of a blockchain-inspired ledger while making it clear that the ledger is simulated.
- Assess the current prototype and identify the improvements required for dependable verification.

3.1 Project scope

Included in current prototype	Outside current scope
Dashboard and sample summary cards	Official government or education registry integration
File-selection interface and document-detail fields	Persistent server-side database and secure cloud storage
Browser-side SHA-256 calculation	Complete candidate-file versus trusted-hash comparison
Reference-based lookup and sample status	Live distributed blockchain or smart contracts
Sample history and simulated ledger indicator	Production identity checks, permissions and security auditing

4. Technologies and concepts

4.1 React, TypeScript and Vite

We used React to split the interface into reusable components, so the dashboard, upload, verification and history pages could be handled separately. TypeScript made the structure of the data easier to follow and helped reduce mistakes while coding. Vite was used as the development and build tool for the front-end application.

4.2 SHA-256 hashing

SHA-256 belongs to the Secure Hash Standard and produces a 256-bit result, normally written as 64 hexadecimal characters. A useful way to understand this value is as a fingerprint made from the exact bytes of a file. If the file is edited, even slightly, its digest should also change.

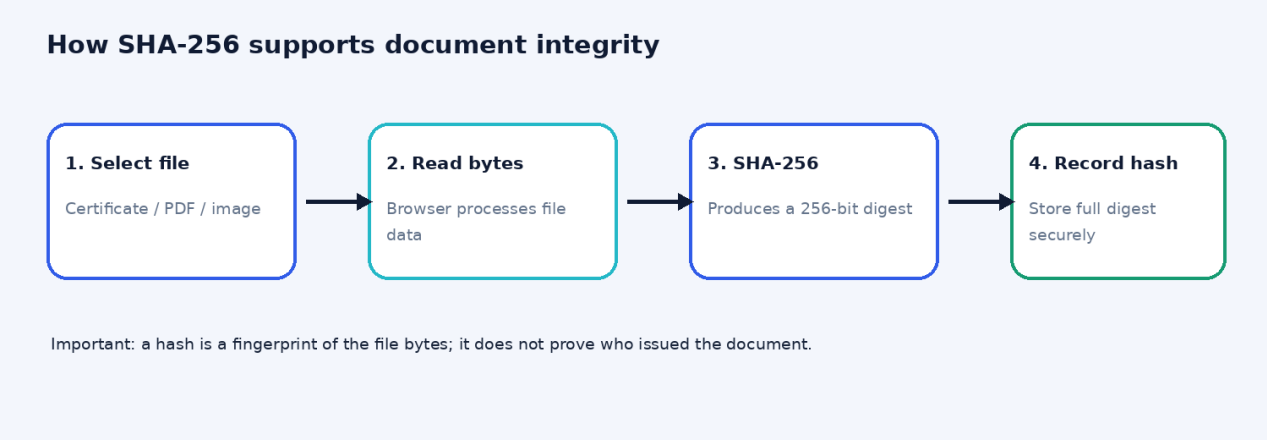


Figure 1. Conceptual hashing workflow. The full digest must be retained and compared for meaningful integrity checking.

A proper integrity check needs both the original document digest and the digest of the file being checked. The original value must be stored safely in a trusted record, and the candidate file must be hashed using the same method. Only then can the two complete values be compared. TrustDoc currently demonstrates the browser hashing stage, but the reference search does not yet complete this full comparison.

4.3 Blockchain-inspired ledger

Blockchain systems link records with cryptographic hashes and use a distributed agreement process to decide which records are accepted. Depending on the design, this can make hidden changes more difficult. Still, a blockchain-themed screen or a visual ledger is not enough to make an application a real blockchain system.

For TrustDoc, the blockchain idea is mainly used to explain integrity and linked records. The version we reviewed does not run across a distributed network and does not include consensus, smart contracts or a live blockchain ledger.

4.4 Cloud services as a future option

If the project is developed further, a managed database and protected file storage could be used for trusted document information. This would bring additional responsibilities, including user permissions, backups, validation, privacy protection and a controlled registration process for authorised issuers. None of these cloud services are connected to the present prototype.

5. System design and workflow

5.1 Prototype architecture

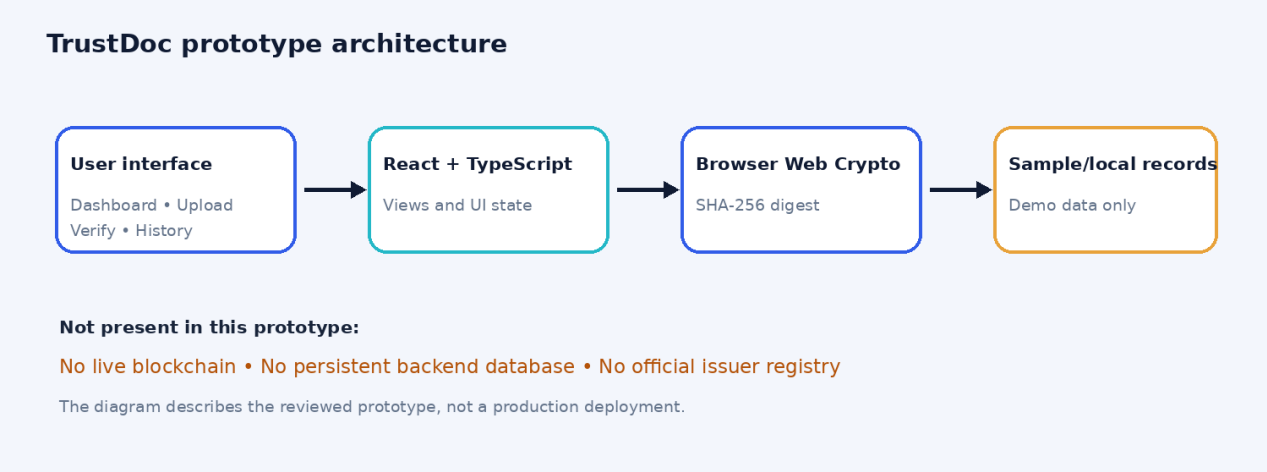


Figure 2. Simplified architecture of the reviewed front-end prototype.

Everything in the current prototype runs through the browser. React and TypeScript control the user-facing pages, and the Web Crypto API is used when a SHA-256 digest is calculated. The records shown by the interface are samples or local data. There is currently no permanent backend, official registry connection or live blockchain.

5.2 Intended registration workflow

1. First, the user visits the certificate-upload page and chooses a supported file.
2. The browser then reads the file bytes and creates its SHA-256 digest.
3. Next, the user adds details such as the document type, holder's name, issue date and issuing authority.
4. In a real system, an authorised issuer would check these details before saving the complete digest in a trusted registry.
5. The prototype currently shows the interface and the hashing idea, but it does not include the permanent registry required for the complete workflow.

5.3 Verification workflow

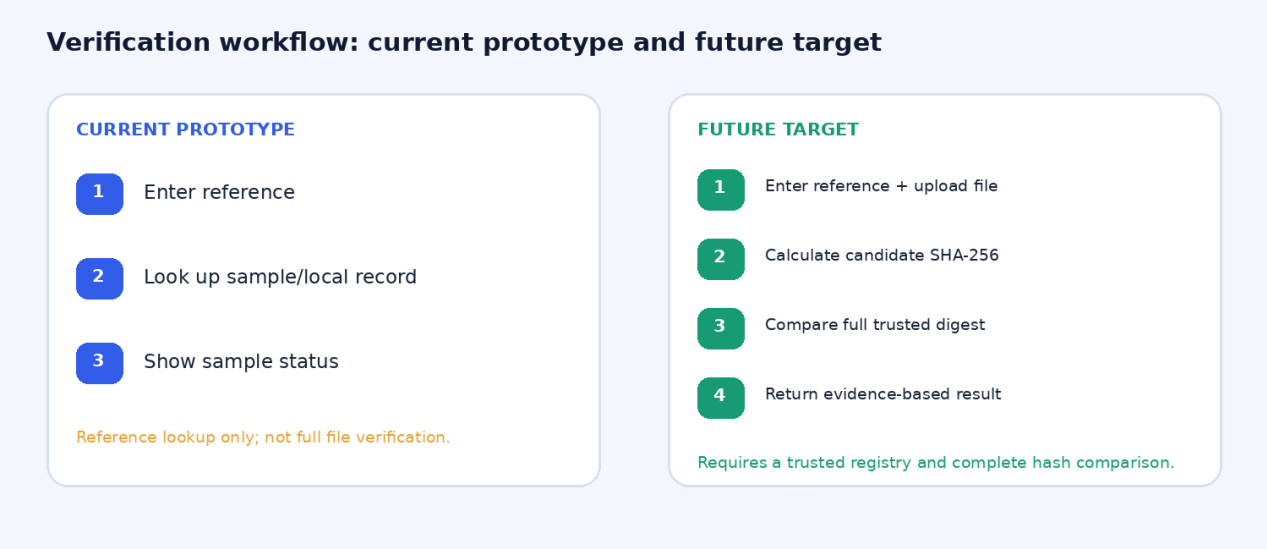


Figure 3. Comparison of the existing reference lookup with a more complete future verification workflow.

At present, the user enters a reference number and TrustDoc searches the available sample or local records. The result shown on the screen therefore depends on that lookup and the demo data. A stronger version would also calculate the submitted file's hash, collect the full trusted digest and compare them before giving an integrity result.

5.4 Expected result language

The result message needs to match the evidence that the system actually has. Until file matching and issuer checks are working, TrustDoc should not state that a document is definitely authentic. Messages such as “Reference found” or “Record located” give a more honest description of the current result.

6. Prototype implementation

Screen / feature	Current behaviour	Important note
Dashboard	Gives a quick view of summary cards, recent document entries and an integrity-related panel.	The numbers and several records are included only as demonstration data.
Certificate upload	Lets the user choose a file, enter document details and calculate the file's SHA-256 digest.	Showing a digest does not mean the document has been registered or approved by an official issuer.
Verify document	Uses a document reference to find and display the related sample or local information.	The candidate file is not yet compared with a complete trusted hash.
Verification history	Shows example rows containing references, results and timestamps from earlier checks.	These rows are examples and are not saved as a permanent audit log.
Ledger and QR-style visual	Helps communicate the intended product idea through the interface.	The ledger is a simulation, and the QR-style image is decorative rather than a standards-compliant code that can be scanned.

6.1 Dashboard

When the application opens, the dashboard gives the user a quick picture of the prototype. The cards make the main values easy to notice, while the recent-documents area suggests how records might be arranged in a real system. Since the current figures are examples, they should not be mistaken for live usage statistics.

6.2 Certificate upload

The upload page helped us demonstrate what information might be collected when a document is registered. A production version would need additional checks: the entered information must be validated, the issuer must be authorised, the full digest must be saved, and the system must record who completed the registration and when it happened.

6.3 Document verification

On the verification page, users can enter a reference and can also see an area intended for a comparison file. That area is part of the interface design, but the actual file comparison has not yet been completed. In the same way, the QR-style image is a placeholder rather than a functioning QR code.

6.4 Verification history

The history page demonstrates one possible way of showing earlier verification attempts. To turn this into a real audit trail, the entries would need permanent storage, trustworthy timestamps, access restrictions and protection from unauthorised editing.

7. Prototype screenshots

The following screenshots come directly from the prototype we reviewed. The comments beside them describe what the user can see, while also noting where the visible design goes beyond the functions that are currently implemented.

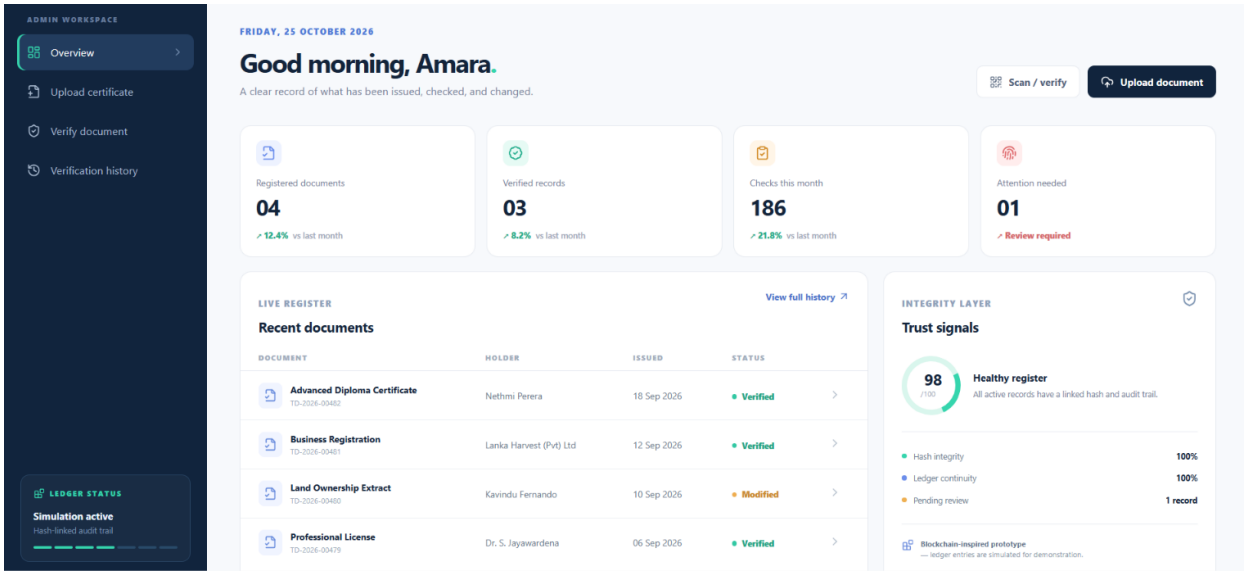


Figure 4. Overview dashboard showing sample summary cards, recent records and the simulated ledger panel.

As a starting screen, the dashboard makes the document-register idea easy to follow. It brings the main information together in one place, although the figures and entries shown are examples rather than data from a running service.

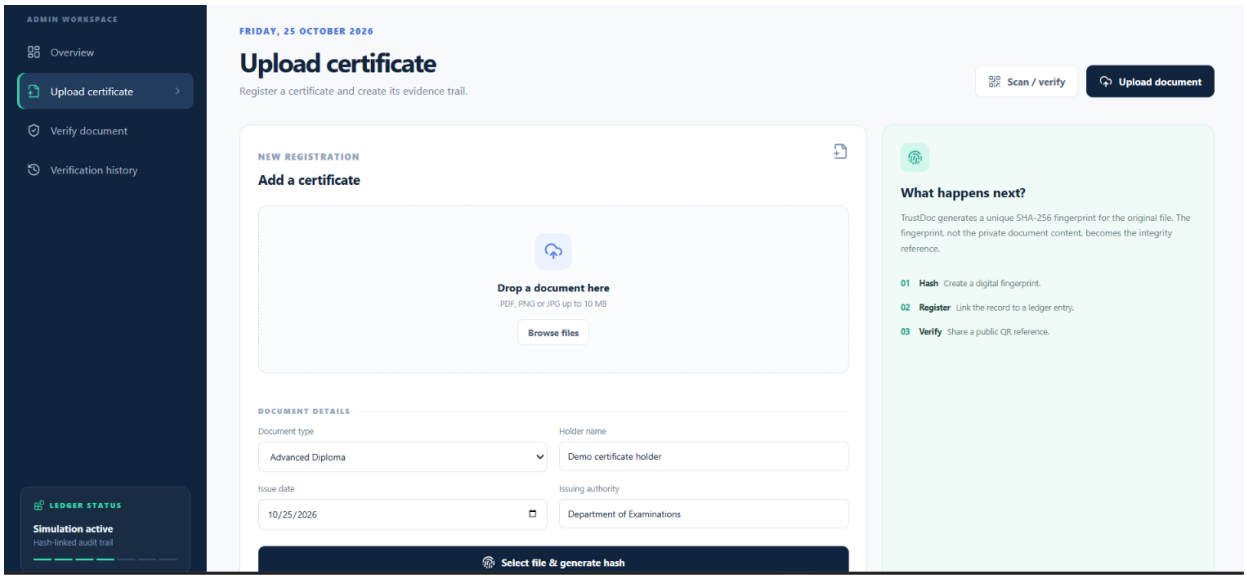


Figure 5. Certificate upload interface with file selection and document-detail fields.

The upload screen shows the main steps a user might follow when registering a document. For actual use, the system would still need to confirm the issuer, validate the document details and place the full hash in a protected record.

7. Prototype screenshots (continued)

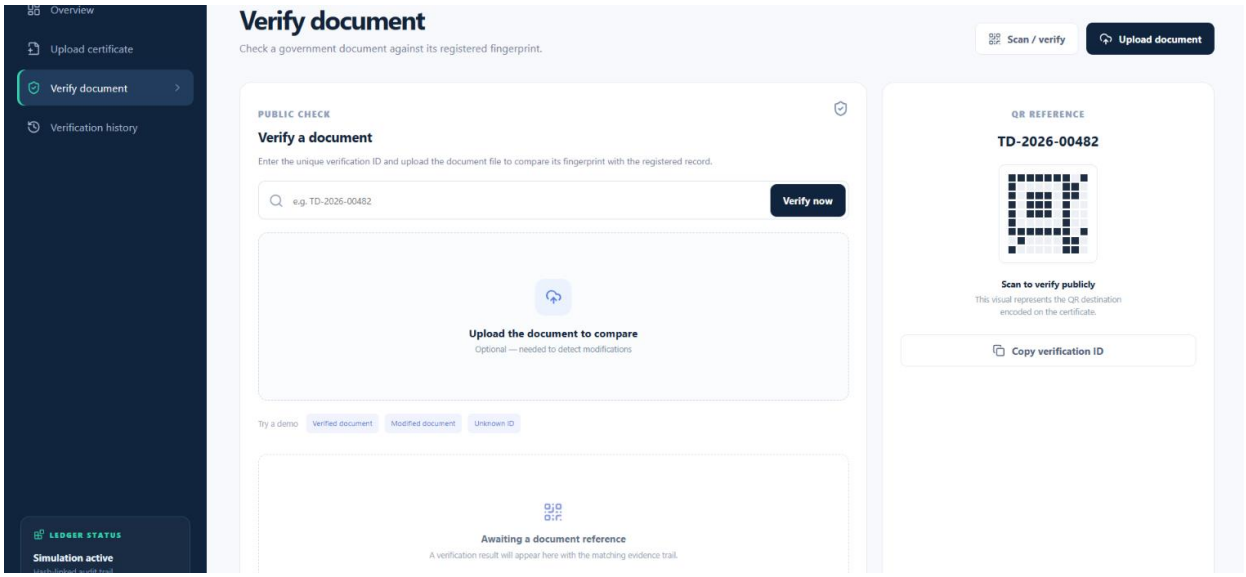


Figure 6. Reference-based verification interface with a comparison-file area and decorative QR-style visual.

The verification screen makes the public-check concept easy to understand, but the current result is produced from sample or local reference data. Because the complete candidate-file hash comparison is not yet performed, the screen cannot prove that a document is authentic.

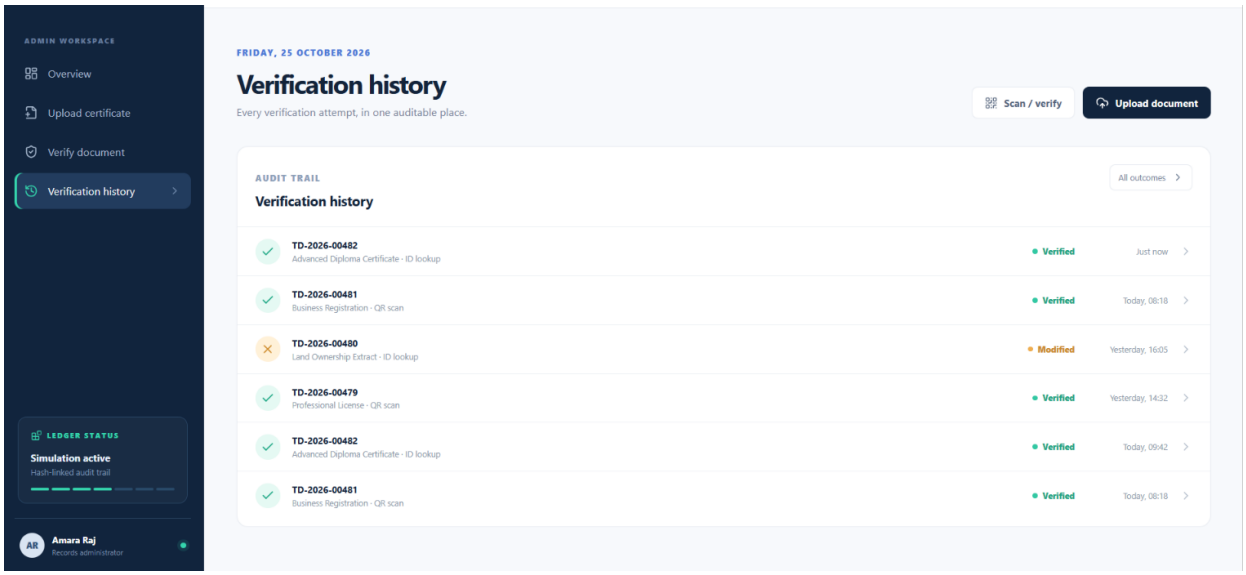


Figure 7. Verification-history screen showing sample audit-style entries.

This page gives a useful example of how previous checks could appear to the user. However, the displayed rows are only sample entries and are not stored as a permanent or tamper-evident audit history.

Screenshot review summary

Observation	Interpretation
Four clear interface areas	Together, the four sections show a complete and easy-to-follow user journey.
Consistent document-checking theme	The consistent visual style makes the purpose of the application

	easier to recognise.
Sample figures and history entries	They make the prototype feel realistic, but they do not represent live activity.
Reference lookup and QR-style graphic	More backend and verification work is needed before the interface can support genuine authenticity claims.

8. Prototype review and limitations

8.1 Review summary

From our review of the source code and screenshots, TrustDoc succeeds in presenting the main document-checking idea as a front-end prototype. The SHA-256 calculation gives users a practical introduction to file fingerprints, and the four pages make the proposed process easy to follow from start to finish. At the same time, it is important not to confuse a convincing interface with a fully verified service.

8.2 Current limitations

- The application is not connected to an official government, education or document-issuer registry.
- There is no permanent backend database or protected cloud storage for documents.
- A candidate file's complete SHA-256 digest is not yet compared with a trusted registered digest.
- The QR-style image is part of the design and is not a standards-compliant QR code that users can scan.
- Some dashboard values and verification-history entries are samples or hard-coded examples.
- The ledger is simulated; the prototype has no distributed blockchain, consensus process or smart contracts.
- Features expected in a production system, such as authentication, user roles, issuer validation, secure deployment and security auditing, have not been implemented.

8.3 Security and trust considerations

When two full hashes match, this can show that the checked file has the same contents as an earlier trusted copy, provided the saved digest is secure and the comparison is correct. Even then, the hash cannot tell us who created the document, whether the issuer had authority, or whether the first record was genuine.

Before a service like this could be used in practice, it would need clear policies for approved issuers, user authentication, stored data, access logs, revocation and disputes. Personal information should be limited to what the verification process genuinely needs and should be protected throughout the system.

8.4 Evaluation approach

Our evaluation focused on the source code and the interface available in the prototype. We did not carry out a formal test plan, an independent security review or a production deployment. For that reason, the screenshots in this report are evidence of the visible pages only, not proof that every suggested function works behind them.

9. Future improvements

Rather than attempting every improvement at once, TrustDoc could be developed in stages. The priorities below start with the functions needed for a meaningful integrity check and then move towards security, trusted issuers and real operational use.

Priority	Improvement	Why it matters
1 — Core integrity	Save the full 64-character SHA-256 digest and compare it with the digest calculated from the file being checked.	This is the main step needed to perform a real file-integrity check.
2 — Trusted records	Develop a persistent database and allow only authorised issuers to add document records through a controlled process.	This would replace temporary demo data with records the system can reliably retrieve later.
3 — Clear results	Use status messages that reflect the actual check, clearly separating “record found”, “hash matched” and “issuer verified”.	Clear wording would help users understand the result without overstating what has been proven.
4 — QR support	Create a standards-compliant QR code that opens the correct online verification record.	This would let a user reach the relevant record quickly from a phone.
5 — Security	Introduce authentication, role-based permissions, input validation, secure connections, audit logs and reliable backups.	Together, these controls would reduce unauthorised access and protect stored records.
6 — Issuer integration	Where an approved interface is available, connect the system to an official registry or an authorised issuing organisation.	This would allow the verification result to come from a recognised and trusted source.
7 — Ledger evaluation	If blockchain remains part of the project, either develop a genuine ledger design and document it properly or clearly present the ledger as a simulation.	Doing this would keep the blockchain description accurate and avoid misleading users.
8 — Operational data	Replace the example dashboard values and history rows with information created by real activity in the application.	The dashboard would then become useful for monitoring the system rather than only demonstrating it.

9.1 Suggested future verification logic

1. Check that the reference uses the expected format and that a matching record is available.
2. Load the complete trusted digest from the protected record.
3. Calculate the candidate file's SHA-256 digest from its exact bytes.
4. Compare the full candidate digest with the full digest saved in the record.
5. Show separate results for the reference lookup, hash comparison and issuer or record status.
6. Store the result in a protected audit log without keeping unnecessary personal information.

10. Conclusion

TrustDoc provides a clear student prototype of a digital document-checking system. The dashboard, certificate-upload page, reference-based verification page and sample history page show how a user could interact with the service. The use of browser-side SHA-256 hashing also introduces an important cryptographic idea for checking file integrity.

At the moment, the project is still a front-end demonstration. It cannot independently confirm the authenticity of government documents, compare a newly uploaded file with a complete trusted digest, connect to an official registry or operate on a live blockchain. These limitations matter because a professional verification service must rely on trusted records and real comparison processes, not only on what is shown in the interface.

The most useful next step would be to develop a trusted registration backend and add complete candidate-file hash comparison. After that, the project could introduce clearer result messages, stronger security controls and integration with authorised issuers. With these changes, TrustDoc could be tested more thoroughly as a practical tool for supporting document-integrity checks.

11. References

National Institute of Standards and Technology (NIST). (2015, updated 2023). Secure Hash Standard (SHS), FIPS PUB 180-4.

https://csrc.nist.gov/pubs/fips/180-4/upd1/final

MDN Web Docs. SubtleCrypto: digest() method.

https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/digest

These references provide the technical background for SHA-256 and the browser cryptography API used in the project. If the coursework guidelines require more sources, they should be added using the referencing style specified by the institute.

12. Group information

Student name	Index number
Bhanuka Hewawasam	KUHNDSE261F-007
Semitha Binuk	KUHNDSE261F-009

Project repository

https://github.com/Bhanuka-Hewawasam/ET_BlockChain.git

The repository contains the source code for the project. This report describes the prototype as it was reviewed and does not present the application as a production-ready document-authentication service.